

Express Middleware

Section: 7 — Express.js Framework Basics (Setup, Routing, Middleware)

Subtopic: 03 — Middleware

Estimated Duration: 3–4 hours

Theory

Practical

Topics covered: What middleware is, the middleware pipeline, built-in middleware (`express.json` , `express.urlencoded` , `express.static`), writing custom middleware, the `next()` function, application-level vs router-level middleware, third-party middleware, error-handling middleware

Learning Objectives — This Subtopic

- Define what middleware is in Express and explain how the middleware pipeline processes requests
- Use built-in middleware (`express.json` , `express.urlencoded` , `express.static`) to parse request bodies and serve static files
- Write custom middleware functions and control execution flow with `next()`
- Distinguish between application-level, router-level, and error-handling middleware
- Implement a centralised error-handling middleware that catches and formats errors consistently
- Integrate third-party middleware packages into an Express application

What is Middleware?

In Express, a **middleware function** is any function that has access to the request object (`req`), the response object (`res`), and the **next** function in the application's request-response cycle. Middleware functions can execute code, modify `req` and `res` , end the request-response cycle, or pass control to the next middleware in the stack.

Every Express application is fundamentally a series of middleware function calls. When a request arrives, Express passes it through a chain of middleware functions — one after another — until one of them sends a response. If no middleware sends a response, the client receives nothing and eventually times out.

The signature of a middleware function looks like this:

middleware-signature.js

```
function myMiddleware(req, res, next) {  
  // Do something with req or res  
  next(); // Pass control to the next middleware  
}
```

Three parameters define a standard middleware function: `req` (the incoming request), `res` (the outgoing response), and `next` (a callback that hands control to the next function in the chain). Calling `next()` is critical — without it, the request stalls.

Analogy: Think of middleware as a series of checkpoints at an airport. A passenger (the request) passes through check-in, security screening, passport control, and boarding — each checkpoint inspects or modifies something, then waves the passenger through to the next one. If any checkpoint finds a problem, it stops the passenger (sends a response) immediately. The order of checkpoints matters: you cannot board before passing security.

The Middleware Pipeline

Express processes middleware in the **exact order** it is registered. This is one of the most important concepts to internalise: the sequence of `app.use()` and route handler calls determines what runs first, second, and third.

Create a new project directory and set up a minimal Express application to experiment with middleware ordering:

```
mkdir middleware-lab && cd middleware-lab
```

```
npm init -y
```

```
npm install express
```

```
middleware-lab/  
├─ node_modules/  
├─ package.json  
├─ package-lock.json  
└─ app.js
```

Observing Execution Order

app.js

```
const express = require('express');  
const app = express();  
  
app.use(function (req, res, next) {  
  console.log('Middleware A');  
  next();  
});  
  
app.use(function (req, res, next) {  
  console.log('Middleware B');  
  next();  
});  
  
app.get('/', function (req, res) {  
  console.log('Route handler');  
  res.send('Hello from the route handler');  
});
```

```
app.listen(3000, function () {  
  console.log('Server running on http://localhost:3000');  
});
```

Run the server and make a request:

```
node app.js
```

Then visit `http://localhost:3000` in your browser, or use a second terminal:

```
curl http://localhost:3000
```

Output:

```
Server running on http://localhost:3000  
Middleware A  
Middleware B  
Route handler
```

The terminal output confirms the execution order: Middleware A runs first, calls `next()`, which triggers Middleware B, which calls `next()`, which triggers the route handler. The route handler sends the response, ending the cycle.

Key Insight: `app.use()` registers middleware that runs for **every** incoming request, regardless of the HTTP method or URL path. Route handlers like `app.get('/')` are themselves middleware — they simply have an additional path-matching condition.

What Happens Without `next()` ?

Remove the `next()` call from Middleware A to observe what happens when the chain breaks:

```
app.js (modified – broken chain)
```

```
const express = require('express');  
const app = express();  
  
app.use(function (req, res, next) {  
  console.log('Middleware A');  
});
```

```
// next() is NOT called
});

app.use(function (req, res, next) {
  console.log('Middleware B');
  next();
});

app.get('/', function (req, res) {
  console.log('Route handler');
  res.send('Hello from the route handler');
});

app.listen(3000, function () {
  console.log('Server running on http://localhost:3000');
});
```

Output:

```
Server running on http://localhost:3000
Middleware A
```

Only “Middleware A” is logged. Middleware B and the route handler never execute. The browser hangs because no response is ever sent. This is the most common middleware mistake — forgetting to call `next()` or send a response.

Warning: Every middleware function must either call `next()` to pass control forward, or send a response with `res.send()`, `res.json()`, `res.end()`, etc. Doing neither causes the request to hang indefinitely. The client will eventually receive a timeout error.

Built-in Middleware

Express ships with three built-in middleware functions. These cover the most common needs: parsing JSON bodies, parsing URL-encoded form data, and serving static files.

`express.json()` — Parsing JSON Request Bodies

When a client sends a POST or PUT request with a JSON body, the raw request body is a stream of bytes.

`express.json()` reads that stream, parses the JSON, and attaches the result to `req.body`.

`app.js`

```
const express = require('express');
const app = express();

// Register the JSON parser middleware
app.use(express.json());

app.post('/api/users', function (req, res) {
  console.log('Received body:', req.body);
  res.json({ message: 'User received', data: req.body });
});

app.listen(3000, function () {
  console.log('Server running on http://localhost:3000');
});
```

Test it by sending a JSON payload with `curl` :

```
curl -X POST http://localhost:3000/api/users -H "Content-Type: application/json" -d '{"name": "Alice", "role": "developer"}'
```

Output:

```
// Terminal (server)
Received body: { name: 'Alice', role: 'developer' }

// Response (client)
{"message": "User received", "data": {"name": "Alice", "role": "developer"}}
```

Without `app.use(express.json())`, `req.body` would be `undefined`. The middleware must be registered **before** any route handlers that need to access `req.body`.

`express.urlencoded()` — Parsing Form Data

HTML forms submit data with the content type `application/x-www-form-urlencoded`. This middleware parses that format into `req.body`.

`app.js`

```
const express = require('express');
const app = express();

app.use(express.json());
app.use(express.urlencoded({ extended: true }));

app.post('/login', function (req, res) {
  console.log('Form data:', req.body);
  res.json({ username: req.body.username });
});

app.listen(3000, function () {
  console.log('Server running on http://localhost:3000');
});
```

Simulate a form submission:

```
curl -X POST http://localhost:3000/login -d "username=alice&password=secret123"
```

Output:

```
// Terminal (server)
Form data: { username: 'alice', password: 'secret123' }

// Response (client)
{"username":"alice"}
```

The `{ extended: true }` option allows the parser to handle nested objects and arrays in form data using the `qs` library. Setting it to `false` uses the built-in `querystring` module, which only supports flat key-value pairs.

Note: It is common practice to register both `express.json()` and `express.urlencoded()` near the top of your application so all route handlers can access parsed request bodies regardless of the content type.

`express.static()` — Serving Static Files

`express.static()` serves files from a directory on disk — HTML pages, CSS stylesheets, images, client-side JavaScript. It takes a directory path and makes every file in that directory accessible via URL.

Create a `public` folder with a simple HTML file:

```
mkdir public
```

```
middleware-lab/  
├─ node_modules/  
├─ public/  
│   └─ index.html  
├─ package.json  
├─ package-lock.json  
└─ app.js
```

```
public/index.html
```

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <title>Static Page</title>  
</head>  
<body>  
  <h1>Served from the public folder</h1>  
</body>  
</html>
```

```
app.js
```



```
const express = require('express');
const path = require('path');
const app = express();

// Serve everything inside the "public" directory
app.use(express.static(path.join(__dirname, 'public')));

app.listen(3000, function () {
  console.log('Server running on http://localhost:3000');
});
```

Run the server and visit `http://localhost:3000/index.html` (or just `http://localhost:3000` — Express serves `index.html` by default for the root path):

```
node app.js
```

Output:

Browser displays:

Served from the public folder

`path.join(__dirname, 'public')` builds an absolute path to the `public` directory. Using `__dirname` ensures the path resolves correctly regardless of which directory you run `node app.js` from. The `path` module was covered in Section 5 (File System Operations).

Writing Custom Middleware

Built-in middleware handles common tasks, but real applications need custom behaviour: logging requests, checking authentication, adding headers, measuring response time, and more. Writing custom middleware follows the same `(req, res, next)` pattern.

Stage 1 A Simple Request Logger

Start with the most basic custom middleware: logging the HTTP method and URL of every incoming request.

app.js

```
const express = require('express');
const app = express();

// Custom logger middleware
function logger(req, res, next) {
  console.log(req.method + ' ' + req.url);
  next();
}

app.use(logger);

app.get('/', function (req, res) {
  res.send('Home page');
});

app.get('/about', function (req, res) {
  res.send('About page');
});

app.listen(3000, function () {
  console.log('Server running on http://localhost:3000');
});
```

node app.js

Visit both `http://localhost:3000/` and `http://localhost:3000/about` :

Output:

```
Server running on http://localhost:3000
GET /
GET /about
```

The `logger` function runs before every route handler. It reads `req.method` and `req.url`, logs them, then calls `next()` to continue the chain. This is the fundamental pattern for all custom middleware.

Stage 2 Adding a Timestamp

Extend the logger to include the current date and time:

app.js (updated logger)

```
function logger(req, res, next) {
  const timestamp = new Date().toISOString();
  console.log('[' + timestamp + '] ' + req.method + ' ' + req.url);
  next();
}
```

Output:

```
[2025-06-15T10:32:07.123Z] GET /
[2025-06-15T10:32:09.456Z] GET /about
```

Stage 3 Measuring Response Time

Middleware can also execute code *after* the response is sent. Attach a listener to the `finish` event on the response object to measure how long the request took:

app.js (updated logger)

```
function logger(req, res, next) {
  const start = Date.now();
  const timestamp = new Date().toISOString();

  res.on('finish', function () {
    const duration = Date.now() - start;
    console.log(
      '[' + timestamp + '] ' +
      req.method + ' ' + req.url + ' ' +
      res.statusCode + ' - ' +
      duration + 'ms'
    );
  });

  next();
}
```

Output:

```
[2025-06-15T10:35:12.789Z] GET / 200 - 3ms
[2025-06-15T10:35:14.012Z] GET /about 200 - 1ms
```

The `res.on('finish')` callback fires after Express has sent the response to the client. By capturing `Date.now()` before and after, you get the total processing time. This pattern is how logging libraries like `morgan` work under the hood.

Application-Level vs Router-Level vs Path-Specific Middleware

Not all middleware needs to run on every request. Express provides several ways to control middleware scope.

Application-Level Middleware

Registered with `app.use()`, this middleware runs for every request to the application. The logger example above is application-level middleware.

Path-Specific Middleware

Pass a path as the first argument to `app.use()` to restrict middleware to a specific URL prefix:

`app.js`

```
const express = require('express');
const app = express();

// This middleware only runs for paths starting with /api
app.use('/api', function (req, res, next) {
  console.log('API request detected: ' + req.url);
  next();
});

app.get('/', function (req, res) {
  res.send('Home page');
});
```

```
app.get('/api/users', function (req, res) {
  res.json([{ id: 1, name: 'Alice' }]);
});

app.get('/api/products', function (req, res) {
  res.json([{ id: 1, name: 'Widget' }]);
});

app.listen(3000, function () {
  console.log('Server running on http://localhost:3000');
});
```

Test both paths:

```
curl http://localhost:3000/
```

```
curl http://localhost:3000/api/users
```

```
curl http://localhost:3000/api/products
```

Output:

```
Server running on http://localhost:3000
API request detected: /users
API request detected: /products
```

Notice that the `GET /` request produces no log output — the middleware is scoped to `/api` and does not execute for the root path. Also notice that `req.url` inside the middleware shows the path *relative to the mount point*: `/users` rather than `/api/users`.

Router-Level Middleware

In Section 7.02 (Routing) you learned about `express.Router()` for modular route organisation. Middleware can be attached to a Router instance in exactly the same way as to the application. The middleware then only applies to routes defined on that router:

app.js

```
const express = require('express');
const app = express();
const adminRouter = express.Router();
```

```
// Middleware scoped to the admin router only
adminRouter.use(function (req, res, next) {
  console.log('Admin area accessed at ' + new Date().toISOString());
  next();
});

adminRouter.get('/dashboard', function (req, res) {
  res.send('Admin Dashboard');
});

adminRouter.get('/settings', function (req, res) {
  res.send('Admin Settings');
});

// Mount the router at /admin
app.use('/admin', adminRouter);

app.get('/', function (req, res) {
  res.send('Public Home');
});

app.listen(3000, function () {
  console.log('Server running on http://localhost:3000');
});
```

```
curl http://localhost:3000/
```

```
curl http://localhost:3000/admin/dashboard
```

Output:

```
Server running on http://localhost:3000
Admin area accessed at 2025-06-15T11:02:33.456Z
```

The `GET /` request triggers no admin middleware log — only requests routed through `adminRouter` are affected.

Attaching Multiple Middleware to a Single Route

You can pass more than one middleware function to a route definition. Express executes them in order, left to right. This is useful for composing behaviour like "authenticate, then authorise, then handle the request."

app.js

```
const express = require('express');
const app = express();

function authenticate(req, res, next) {
  // Simulate checking for an API key in the query string
  if (req.query.apiKey === 'abc123') {
    req.user = { name: 'Alice', role: 'admin' };
    console.log('Authenticated: ' + req.user.name);
    next();
  } else {
    res.status(401).json({ error: 'Invalid API key' });
  }
}

function authorise(req, res, next) {
  if (req.user && req.user.role === 'admin') {
    console.log('Authorised: admin access granted');
    next();
  } else {
    res.status(403).json({ error: 'Forbidden' });
  }
}

app.get('/admin/data', authenticate, authorise, function (req, res) {
  res.json({ secret: 'Top secret admin data' });
});

app.listen(3000, function () {
  console.log('Server running on http://localhost:3000');
});
```

Test with and without the correct API key:

```
curl "http://localhost:3000/admin/data?apiKey=abc123"
```

Output:

```
// Terminal
Authenticated: Alice
Authorised: admin access granted

// Response
{"secret":"Top secret admin data"}
```

```
curl http://localhost:3000/admin/data
```

Output:

```
{"error":"Invalid API key"}
```

When the API key is missing, `authenticate` sends a 401 response and does **not** call `next()`. The `authorise` function and route handler never execute. This is the standard pattern for protecting routes: each middleware acts as a gate.

Note: In production, you would not pass API keys as query parameters — this is simplified for demonstration. Authentication and security are covered in detail in Section 13.

Third-Party Middleware

The Express ecosystem includes a large collection of middleware packages published on NPM. Instead of writing everything from scratch, you install a package and register it with `app.use()`. Here is a commonly used example: `morgan`, a HTTP request logger.

```
npm install morgan
```

```
app.js
```

```
const express = require('express');
const morgan = require('morgan');
const app = express();

// Use morgan with the 'dev' format
```



```
app.use(morgan('dev'));

app.get('/', function (req, res) {
  res.send('Home page');
});

app.get('/about', function (req, res) {
  res.send('About page');
});

app.listen(3000, function () {
  console.log('Server running on http://localhost:3000');
});
```

```
node app.js
```

Make requests to both routes:

Output:

```
Server running on http://localhost:3000
GET / 200 2.145 ms - 9
GET /about 200 0.312 ms - 10
```

`morgan('dev')` produces a concise, colour-coded log line for each request showing the method, path, status code, response time, and content length. The `'dev'` format is ideal for development; other formats like `'combined'` produce Apache-style access logs suitable for production.

This is functionally equivalent to the custom logger middleware built earlier — but `morgan` is battle-tested, configurable, and supports output to files and streams.

Error-Handling Middleware

Express treats middleware with **four parameters** as error-handling middleware. The signature is `(err, req, res, next)` — the additional first parameter `err` receives the error object. This is how Express distinguishes error handlers from regular middleware.

Stage 1 How Errors Reach the Handler

There are two ways to trigger error-handling middleware:

1. **Calling `next(err)`** — passing any value to `next()` tells Express to skip all remaining regular middleware and jump to the first error handler.
2. **Throwing an error inside a synchronous handler** — Express catches synchronous throws automatically.

app.js

```
const express = require('express');
const app = express();

app.get('/error-demo', function (req, res, next) {
  // Simulate something going wrong
  const err = new Error('Something broke');
  err.status = 500;
  next(err); // Pass the error to the error handler
});

// Error-handling middleware (MUST have 4 parameters)
app.use(function (err, req, res, next) {
  console.error('Error caught:', err.message);
  const statusCode = err.status || 500;
  res.status(statusCode).json({
    error: {
      message: err.message,
      status: statusCode
    }
  });
});

app.listen(3000, function () {
  console.log('Server running on http://localhost:3000');
});
```

```
curl http://localhost:3000/error-demo
```

Output:

```
// Terminal
Error caught: Something broke

// Response
{"error":{"message":"Something broke","status":500}}
```

The route handler creates an `Error` object, attaches a `status` property, and passes it to `next()`. Express skips any remaining regular middleware and invokes the error handler, which logs the error and sends a JSON response to the client.

Important: Error-handling middleware **must** declare exactly four parameters (`err`, `req`, `res`, `next`), even if you do not use `next`. Express uses the function's `.length` property (the number of declared parameters) to identify error handlers. If you write (`err`, `req`, `res`) with only three parameters, Express will treat it as regular middleware and it will not catch errors.

Stage 2 Placing the Error Handler Correctly

Error-handling middleware must be registered **after** all other `app.use()` calls and route definitions. Express only forwards errors to handlers that appear later in the middleware stack.

app.js

```
const express = require('express');
const app = express();

app.use(express.json());

// --- All routes go here ---
app.get('/', function (req, res) {
  res.send('Home page');
});

app.get('/fail', function (req, res, next) {
  next(new Error('Intentional failure'));
});

// --- 404 handler (no matching route found) ---
app.use(function (req, res, next) {
  const err = new Error('Not Found');
  err.status = 404;
  next(err);
});
```

```
});  
  
// --- Error handler (MUST be last) ---  
app.use(function (err, req, res, next) {  
  const statusCode = err.status || 500;  
  res.status(statusCode).json({  
    error: {  
      message: err.message,  
      status: statusCode  
    }  
  });  
});  
  
app.listen(3000, function () {  
  console.log('Server running on http://localhost:3000');  
});
```

```
curl http://localhost:3000/nonexistent
```

Output:

```
{"error":{"message":"Not Found","status":404}}
```

```
curl http://localhost:3000/fail
```

Output:

```
{"error":{"message":"Intentional failure","status":500}}
```

The structure here is the standard Express pattern: routes first, then a 404 catch-all, then the error handler at the very end. The 404 middleware fires when no route matches — it creates an error and forwards it. Both the 404 and the `/fail` route converge on the same error handler, giving you a single place to format error responses consistently.

Stage 3 Handling Async Errors

Express does **not** automatically catch errors thrown inside asynchronous code (callbacks, promises). You must catch them yourself and pass them to `next()` :

```
app.js (async route)
```

```
const express = require('express');
const fs = require('fs');
const app = express();

app.get('/read-file', function (req, res, next) {
  fs.readFile('nonexistent.txt', 'utf8', function (err, data) {
    if (err) {
      err.status = 500;
      return next(err); // Forward the error
    }
    res.send(data);
  });
});

// Error handler
app.use(function (err, req, res, next) {
  const statusCode = err.status || 500;
  res.status(statusCode).json({
    error: {
      message: err.message,
      status: statusCode
    }
  });
});

app.listen(3000, function () {
  console.log('Server running on http://localhost:3000');
});
```

```
curl http://localhost:3000/read-file
```

Output:

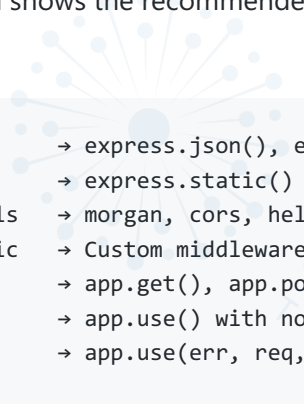
```
{"error":{"message":"ENOENT: no such file or directory, open 'nonexistent.txt',"status":500}}
```

The `fs.readFile` callback follows the Node.js error-first convention (covered in Section 3). When the file does not exist, the callback receives an error. The code checks for the error and passes it to `next(err)` instead of crashing the application.

Key Insight: Starting with Express 5 (currently in beta), Express automatically catches rejected promises returned from route handlers and forwards them to the error handler. In Express 4 (the current stable release), you must handle async errors manually with try-catch or by calling `next(err)`.

Middleware Registration Order — A Complete Picture

The following diagram shows the recommended order of middleware registration in a typical Express application:

- 
1. Body parsers → `express.json(), express.urlencoded()`
 2. Static files → `express.static()`
 3. Third-party utils → `morgan, cors, helmet, etc.`
 4. Application logic → Custom middleware (auth, logging, etc.)
 5. Route handlers → `app.get(), app.post(), router.use()`
 6. 404 catch-all → `app.use()` with no path match
 7. Error handler → `app.use(err, req, res, next)`

This order ensures that body parsing is complete before any route handler accesses `req.body`, static files are served without hitting application logic, and the error handler is the final safety net for anything that goes wrong.

Summary

Concept	Key Point
Middleware function	A function with <code>(req, res, next)</code> that processes requests in a pipeline

Concept	Key Point
<code>next()</code>	Passes control to the next middleware; without it, the request hangs
<code>express.json()</code>	Parses incoming JSON request bodies into <code>req.body</code>
<code>express.urlencoded()</code>	Parses URL-encoded form data into <code>req.body</code>
<code>express.static()</code>	Serves files from a directory (CSS, images, HTML) with no route handler needed
Custom middleware	Any <code>(req, res, next)</code> function registered with <code>app.use()</code> or passed directly to routes
Path-specific middleware	<code>app.use('/api', fn)</code> restricts middleware to requests matching that path prefix
Router-level middleware	Middleware attached to an <code>express.Router()</code> instance, scoped to that router's routes
Multiple middleware per route	<code>app.get('/path', fn1, fn2, handler)</code> chains middleware for a specific route
Error-handling middleware	Uses four parameters <code>(err, req, res, next)</code> ; must be registered last
Async error handling	In Express 4, async errors must be caught and forwarded with <code>next(err)</code> manually

Interview Questions

1. What is middleware in Express, and what three arguments does a standard middleware function receive?

Expected answer: Middleware is a function that sits in the request-response pipeline. It receives the

request object (`req`), the response object (`res`), and a `next` callback. It can modify `req` / `res` , end the cycle by sending a response, or call `next()` to pass control to the next middleware in the stack.

2. What happens if a middleware function neither calls `next()` nor sends a response?

Expected answer: The request hangs indefinitely. The client never receives a response and will eventually time out. Every middleware must either call `next()` or send a response to prevent this.

3. How does Express distinguish error-handling middleware from regular middleware?

Expected answer: Error-handling middleware declares four parameters: (`err`, `req`, `res`, `next`) . Express checks the function's `.length` property. If it is 4, Express treats it as an error handler. This means you must declare all four parameters even if you do not use `next` .

4. Explain the difference between `next()` and `next(err)` .

Expected answer: Calling `next()` with no arguments passes control to the next regular middleware in the stack. Calling `next(err)` with an argument skips all remaining regular middleware and jumps directly to the first error-handling middleware.

5. Why should `express.json()` be registered before route handlers?

Expected answer: Because middleware executes in registration order. If `express.json()` is registered after a route handler, that handler will see `req.body` as `undefined` because the JSON parser has not yet run.

6. What does `express.static()` do, and why is `path.join(__dirname, 'public')` recommended over a relative path?

Expected answer: `express.static()` serves files directly from a specified directory. Using `path.join(__dirname, 'public')` creates an absolute path based on the file's location on disk, ensuring the path resolves correctly regardless of the working directory from which `node` was invoked.

7. How can you apply middleware to only a subset of routes?

Expected answer: Three approaches: pass a path prefix to `app.use('/api', fn)` , attach the middleware to a specific `express.Router()` instance, or pass it directly to individual route definitions such as `app.get('/path', authMiddleware, handler)` .

8. In Express 4, does Express automatically catch errors thrown inside asynchronous callbacks?

How do you handle this?

Expected answer: No. Express 4 only catches synchronous throws. For asynchronous errors (callbacks, promises), you must catch the error yourself and call `next(err)` to forward it to the error-handling middleware. Express 5 adds automatic promise rejection handling.

Practical Assignment: Build a Middleware Pipeline

Objective: Create an Express application that demonstrates a complete middleware pipeline — including a custom logger, a request timestamp injector, body parsing, route handling, a 404 handler, and centralised error handling.

1. Create a new project directory called `middleware-pipeline` and initialise it with `npm init -y` and `npm install express`.
2. Create `app.js` with the following middleware, registered in this exact order:
 1. **Request Logger** — a custom middleware that logs the timestamp, HTTP method, URL, and a unique request ID (use `Date.now()` for a simple ID). Attach the request ID to `req.requestId` so later middleware can reference it.
 2. **Body Parsers** — `express.json()` and `express.urlencoded({ extended: true })`.
 3. **Static Files** — serve a `public` directory. Create a `public/index.html` file with a simple heading.
3. Define these routes:
 1. `GET /api/health` — returns `{ status: "ok", requestId: req.requestId }`
 2. `POST /api/echo` — returns whatever JSON the client sends as `{ echo: req.body, requestId: req.requestId }`
 3. `GET /api/error` — creates an `Error` with message "Test error" and status 503, then passes it to `next()`

4. Add a **404 catch-all** middleware after all routes that creates a "Not Found" error with status 404 and forwards it to the error handler.
5. Add an **error-handling middleware** as the final middleware. It should log the error message along with `req.requestId`, and respond with JSON: `{ error: { message, status, requestId } }`.
6. Test every path with `curl` and verify the outputs match the expected results below.

Expected output:

```
curl http://localhost:3000/api/health
```

Output:

```
// Terminal log
[2025-06-15T12:00:01.234Z] GET /api/health (id: 1750070401234)

// Response
{"status":"ok","requestId":1750070401234}
```

```
curl -X POST http://localhost:3000/api/echo -H "Content-Type: application/json" -d
"{\"message\": \"hello\"}"
```

Output:

```
// Terminal log
[2025-06-15T12:00:05.678Z] POST /api/echo (id: 1750070405678)

// Response
{"echo":{"message":"hello"},"requestId":1750070405678}
```

```
curl http://localhost:3000/api/error
```

Output:

```
// Terminal log
[2025-06-15T12:00:08.901Z] GET /api/error (id: 1750070408901)
ERROR [1750070408901]: Test error

// Response
{"error":{"message":"Test error","status":503,"requestId":1750070408901}}
```

```
curl http://localhost:3000/nonexistent
```

Output:

```
// Terminal log
[2025-06-15T12:00:12.345Z] GET /nonexistent (id: 1750070412345)
ERROR [1750070412345]: Not Found

// Response
{"error":{"message":"Not Found","status":404,"requestId":1750070412345}}
```

Bonus Challenge: Add a `responseTime` field to every JSON response. Measure the time elapsed between when the request arrives at the logger middleware and when the route handler sends its response. Hint: capture `Date.now()` at the start and compute the difference just before calling `res.json()`.